

However, notice the Python code for the `n` is even recursive case:

Python

```
--snip--
    elif n % 2 == 0:
        # RECURSIVE CASE (when n is even)
        result = exponentByRecursion(a, n / 2)
        return result * result
--snip--
```

And notice the equivalent JavaScript recursive case:

JavaScript

```
--snip--
    } else if (n % 2 === 0) {
        // RECURSIVE CASE (when n is even)
        result = exponentByRecursion(a, n / 2);
        return result * result;
--snip--
```

This recursive case does not have the recursive call as its last action. We could get rid of the `result` local variable, and instead call the recursive function twice. This would reduce the recursive case to the following:

```
--snip--
return exponentByRecursion(a, n / 2) * exponentByRecursion(a, n / 2)
--snip--
```

However, now we have two recursive calls to `exponentByRecursion()`. Not only does this needlessly double the amount of computation the algorithm performs, but the last action performed by the function is to multiply the two return values. This is the same problem the recursive Fibonacci algorithm has: if the recursive function has multiple recursive calls, at least one of those recursive calls can't be the last action of the function.

Tail Recursive Odd-Even

To determine whether an integer is odd or even, you can use the `%` modulus operator. The expression `number % 2 == 0` will be `True` if `number` is even, and `False`